

Лабораторна робота №3.

Вивчення компоненти TCPWM_PWM_PDL мікроконтролера PSoC 6.

Мета роботи: Ознайомитися з компонентами TCPWM_PWM_PDL, TCPWM_Counter_PDL мікроконтролерів PSoC 6; основними їх застосуваннями та параметрами налаштування. Реалізувати проекти в IDE PSoC Creator 4.4 з використанням компонент PWM, Timer, Counter.

Теоретичні відомості.

Основи широтно-імпульсної модуляції (ШІМ).

Визначення широтно-імпульсної модуляції вже міститься в самій назві - модуляція тільки ширини імпульсу, а не частоти.

У цифровій системі керування ШІМ використовується для керування потужністю, що подається на пристрій. Таким чином, ШІМ є дуже ефективним методом керування аналоговими пристроями за допомогою цифрових виходів мікроконтролера. ШІМ широко використовується в багатьох областях, таких як вимірювання, зв'язок, управління потужністю та перетворення.

ШІМ сигнал складається з двох основних параметрів, які визначають його поведінку: коефіцієнту заповнення (робочий цикл) та частоти (рис. 3.1).

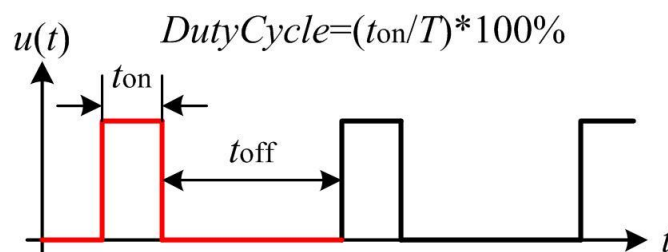


Рис. 3.1. Вигляд ШІМ сигналу

Робочий цикл описує інтервал часу, на протязі якого сигнал перебуває у високому (увімкненому) стані, як відсоток від загального часу, необхідного для завершення одного циклу. Менший робочий цикл відповідає меншій потужності, оскільки живлення більшу частину часу відключено. Робочий цикл виражається у відсотках, 100% робочий цикл буде повністю включений так само, як і при підключенні сигналу $u(t)$ до напруги живлення V_{cc} (високий стан). 0% робочого циклу буде таким самим, як заземлення сигналу.

Частота визначає, як швидко ШІМ завершує цикл. Наприклад, 100 Гц – це 100 циклів за секунду. Іншими словами, частота показує, як швидко ШІМ перемикається між високим (логічна "1") і низьким (логічний "0") станами.

У цифровій системі ШІМ - це метод отримання змінної напруги за допомогою цифрових засобів. Як правило, цифрова система має тільки дві

вихідні напруги: високу (5 В, 3,3 В ... тощо) або низьку (0 В). Але як цифрова система може виробляти напругу, яка знаходиться між високою та низькою напругами?

Для пояснення цього скористаємося простими математичними виразами для аналізу ШІМ сигналу і середнього вихідного сигналу.

Математичний аналіз ШІМ сигналу.

ШІМ використовує прямокутну імпульсну хвилю, ширина імпульсу якої модулюється. Це призводить до зміни середнього значення форми хвилі. Розглянемо форму сигналу, зображеного на рис. 3.2.

Імпульсний сигнал описується функцією $f(t)$ з періодом T та коефіцієнтом заповнення D (0.0 – 1.0). Середня напруга форми хвилі визначається виразом:

$$\bar{y} = \frac{1}{T} \int_0^T f(t) dt. \quad (3.1)$$

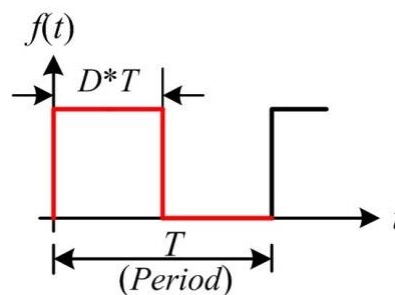


Рис. 3.2. Прямокутна імпульсна хвиля ШІМ сигналу.

Так як $f(t)$ – імпульсна хвиля, то розглянемо один період T . Додатний імпульс відповідає інтервалу $0 < t < D \times T$, а нульовий імпульс – $D \times T < t < T$. Підставимо ці величини в вираз (3.1). Отримаємо:

$$\bar{y} = \frac{1}{T} \left(\int_0^{D \cdot T} 1 dt + \int_{D \cdot T}^T 0 dt \right) = \frac{1}{T} [DT * 1 + T(1 - D) * 0] = D * 1$$

Звідси бачимо, що середнє значення сигналу знаходиться в прямій залежності від коефіцієнта заповнення D .

Наприклад, якщо цифровий сигнал створює імпульс високого рівня (5 В) і низького рівня (0 В) на протязі однакового часу, то коефіцієнт заповнення становить 50%. Тому середня напруга складатиме 2,5 вольт. Тепер нехай напруга у високому стані перебуває на протязі 7 мксек і в низькому стані на протязі 3 мксек. Коефіцієнт заповнення тепер становить 70%, а середня напруга імпульсного сигналу складає 70% від 5 вольт або $5 \text{ В} * 0,7 = 3,5 \text{ В}$.

Типи ШІМ.

Існує кілька типів ШІМ. Їх можна класифікувати різними способами.

Класифікація ШІМ сигналів на симетричні та асиметричні ШІМ сигнали.

1. Симетричні сигнали ШІМ:

- Імпульси симетричних ШІМ сигналів завжди симетричні відносно центру кожного періоду ШІМ (рис. 3.3).

- Симетричні ШІМ сигнали часто використовуються для трифазних асинхронних двигунів змінного струму та беколектронних двигунів постійного струму через нижчі гармонічні спотворення, які генеруються фазними струмами порівняно з асиметричними ШІМ сигналами.

2. Асиметричні ШІМ сигнали:

- Імпульси асиметричного сигналу ШІМ завжди мають одну сторону, яка вирівняна з одним кінцем кожного періоду ШІМ (рис. 3.3).

- Асиметричні ШІМ сигнали можна використовувати для крокових двигунів та інших двигунів із змінним реактивним реактивом.

Класифікація ШІМ сигналу по його вирівнюванню (рис. 3.3):

1. Вирівняний по центру ШІМ сигнал:

- симетричний ШІМ сигнал;
- ШІМ сигнали з вирівнюванням по центру найчастіше використовуються при управлінні двигунами змінного струму для підтримки вирівнювання фаз.

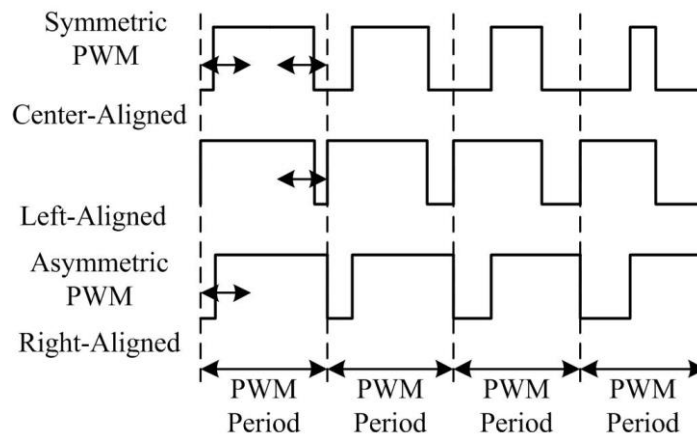


Рис. 3.3. Три типи ШІМ сигналу: ШІМ з вирівнюванням по центру, ШІМ з вирівнюванням по лівому краю, ШІМ з вирівнюванням по правому краю

2. ШІМ сигнали з вирівнюванням по лівому краю:

- асиметричні ШІМ сигнали;
- ШІМ сигнали з вирівнюванням по лівому краю використовуються для більшості універсальних застосувань ШІМ.

3. ШІМ сигнали з вирівнюванням по правому краю:

- асиметричні ШІМ сигнали;
- ШІМ сигнали з вирівнюванням по правому краю в основному використовуються тільки в особливих випадках, коли потрібне вирівнювання протилежне до ШІМ сигналу з вирівнюванням по лівому краю.

4. ШІМ сигнали з двома фронтами:

- ШІМ сигнали з двома фронтами оптимізовані для перетворення енергії, де потрібно відрегулювати фазове вирівнювання.

Генерація ШІМ сигналів за допомогою мікроконтролера з використанням таймера/лічильника.

Основна ідея генерації ШІМ сигналу полягає у використанні лічильника (або таймера), величини СМР (compare - порівняння) і цифрового виходу мікроконтролера (рис. 3.4). Лічильник (сумуючий або віднімаючий) неперервно підраховує і порівнює зі значенням СМР. Цифровий вихід (ШІМ) буде змінено, коли величина, підрахована лічильником, збігається зі значенням СМР або коли лічильник скидається. ШІМ сигнал може бути реалізований програмно або апаратно. Більшість мікроконтролерів мають апаратні модулі, які можуть генерувати ШІМ сигнали після ініціалізації регістрів.

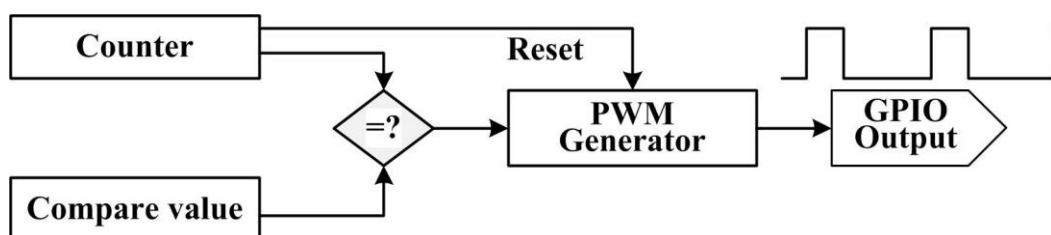


Рис. 3.4. Структурна схема генерації ШІМ сигналів

Основні характеристики компоненти TCPWM_PWM_PDL:

- 16-ти або 32-х бітний лічильник.
- Два програмованих регістри періоду, які можна міняти місцями.
- Два вихідних регістри порівняння, які можна міняти місцями при переповненні і/або до переповнення.
- Режими вирівнювання по лівому краю, вирівнювання по правому краю, вирівнювання по центрі і асиметричне вирівнювання.
- Режими неперервного або однократного виконання.
- Псевдовипадковий режим.
- Два ШІМ- виходи з вставкою "мертвого" часу (Dead Time) та програмованою полярністю.
- Переривання та вихід при переповненні, переході через нуль або порівнянні.
- Входи старт (Start), перезавантаження (Reload), очистки (Kill), обміну (Swap) та підрахунку (Count).
- Для деяких додатків декілька компонент можуть бути синхронізовані разом, наприклад, управління трифазним двигуном.

Загальний опис.

Компонента TCPWM_PWM_PDL є графічним об'єктом, що конфігурується, створеним як верхній рівень драйвера су_tcpwm, доступного в PDL. Вона допускає з'єднання на основі схеми і апаратну конфігурацію, яка визначена в діалоговому вікні "Component Configure".

Компонента TCPWM_PWM_PDL є обгорткою навколо апаратного елементу TCPWM, яка дозволяє налаштовувати апаратний елемент TCPWM для функції ШІМ. З допомогою неї можна синтезувати довільні цифрові сигнали та контролювати робочий цикл (шпаруватість) і період вихідного сигналу TCPWM_PWM_PDL. Компонента TCPWM_PWM_PDL також забезпечує додатковий вихід з можливістю вставки мертвого часу між двома виходами.

Компонента TCPWM_PWM_PDL має декілька параметрів вирівнювання: по лівому краю, по правому краю, по центру і асиметричне. У всіх режимах період та величина, що порівнюється, можуть бути поміняні місцями для створення сигналу довільної форми. Компонента також має псевдовипадковий режим роботи.

Компонента TCPWM_PWM_PDL використовується у випадках, коли потрібно на її виході отримати імпульси прямокутної форми з певним періодом та робочим циклом. Наприклад:

- управління швидкістю обертання двигунів постійного струму;
- управління світлодіодами (LED), зміна яскравості їх свічення.

Швидкий старт роботи з компонентою TCPWM_PWM_PDL.

1. Перетягнути компоненту TCPWM_PWM_PDL з папки Cypress/Digital/Function/ в каталозі компонент на схему проекту (екземпляр цієї компоненти приймає ім'я PWM_1).

2. Двічі клацнути на екземплярі компоненти, щоб відкрити діалогове вікно налаштування (Configure) і налаштувати параметри компоненти.

3. В цій компоненті немає внутрішніх джерел тактових імпульсів. Тому потрібно подати на вхід TCPWM_PWM_PDL тактові імпульси з зовнішнього джерела. В компоненті TCPWM_PWM_PDL доступна функція попереднього масштабування тактової частоти, щоб додатково виконати ділення тактової частоти.

4. Побудувати проект. Для того, щоб перевірити правильність проекту, потрібно додати необхідні модулі PDL в Workspace Explorer та згенерувати задану конфігурацію для екземпляра PWM_1.

5. В файлі main.c ініціалізувати периферійні пристрої та запустити додаток за допомогою API драйверів та макросів препроцесора компонентів.

Якщо не використовується жоден вхідний вивід (наприклад, start), для початку підрахунку потрібно викликати програмну подію Cy_TCPWM_TriggerStart або Cy_TCPWM_TriggerReloadOrIndex. Наприклад:

```
(void) Cy_TCPWM_PWM_Init(PWM_1_HW, PWM_1_CNT_NUM, &PWM_1_config);  
Cy_TCPWM_Enable_Multiple(PWM_1_HW, PWM_1_CNT_MASK);  
Cy_TCPWM_TriggerStart(PWM_1_HW, PWM_1_CNT_MASK);
```

Виконати компіляцію та запрограмувати мікроконтролер PSoC 6 станду. Зовнішній вигляд символу компоненти PWM приведено на рис. 3.5.

Вхідні/вихідні виводи компоненти.

Розглянемо входи та виходи компоненти TCPWM_PWM_PDL. Зірочка (*) у наведеному нижче списку вказує, що вони можуть не відображатися на символі компоненти за умов, приведених в описі цього вводу/виводу.

- clock (Digital Input) - вхід тактових імпульсів, визначає робочу частоту цієї компоненти. Період виводу PWM дорівнює $(1/\text{тактова частота}) \cdot \text{період}$.

- swap [1]* (Digital Input) – цей вхід фіксує команду SWAP. Він є видимим тільки у випадку, якщо для параметру SwapInput встановлено значення, відмінне від disabled. Обмін не відбувається до наступної події ov/un. Потрібно переконатися, що період замінюється тільки при настанні події OV.

- reload[1]* (Digital Input) - цей вхід виконує перезавантаження PWM та запускає PWM. Вхід видимий лише в тому випадку, якщо для параметра ReloadInput встановлено значення, відмінне від disabled. У режимі вирівнювання по лівому краю, лічильник ініціалізується значенням "0". Для режимів центрування та асиметричного вирівнювання лічильник ініціалізується значенням "1". У режимі вирівнювання по правому краю лічильник ініціалізується значенням періоду. Потрібно використовувати тільки тоді, коли лічильник не працює.

- count* (Digital Input) - цей вхід змушує PWM підраховувати залежно від того, як він налаштований. Наприклад, якщо цей вхід налаштований як чутливий до рівня, ШІМ змінюватиме свій підрахунок на кожний попередньо масштабований фронт тактових імпульсів. Цей вхід є видимим, лише тоді, якщо для параметра CountInput встановлено значення, відмінне від disabled. Не використовується і не відображається для режиму Pseudo Random PWM.

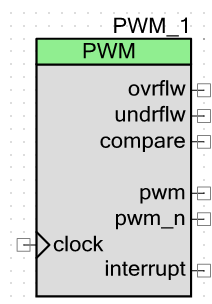


Рис. 3.5. Зовнішній вигляд символу компоненти PWM

- start [1]* (Digital Input) - цей вхід запускає PWM. Він видимий тільки в тому випадку, якщо для параметра StartInput встановлено значення, відмінне від disabled. Використовується лише тоді, коли лічильник не працює.

- kill [1]* (Digital Input) – цей вхід зупиняє PWM при виборі режиму Kill. Він видимий тільки в тому випадку, якщо для параметра KillInput встановлено значення, відмінне від disabled або Stop on Kill.

- ovrflw* (Digital Output) – цей вихід компоненти приймає значення логічної "1", коли значення лічильника переполюється від величини періоду до нуля.

Перезавантаження також викличе переповнення при вирівнюванні по лівому краю. Вихід не використовується і невидимий для псевдовипадкового режиму PWM.

- `undrflw*` (Digital Output) - цей вихід приймає значення логічної "1", коли значення лічильника переходить з нуля до значення періоду. Перезавантаження також генерує цю подію в режимах правого, центрального та асиметричного вирівнювань. Вихід не застосовується і не відображається для псевдовипадкового режиму PWM.

- `compare` (Digital Output) - цей вихід приймає значення логічної "1", коли величина порівняння рівна значенню підрахунку (ця подія генерується, коли повинно відбутися співпадіння (COUNTER не рівний COMPARE і змінюється на COMPARE, в режимах вирівнювання по лівому краю або по правому краю) або коли співпадіння не повинно відбутися (COUNTER рівний COMPARE і змінюється на інше значення в режимах центрування/асиметричного вирівнювання)).

- `pwm` (Digital Output) – вихід PWM.
- `pwm_n` (Digital Output) – інверсний вихід PWM.
- `interrupt` (Digital Output) – цей вихід можна під'єднати тільки до переривання.

Параметри компоненти.

Діалогове вікно "Налаштування компоненти" `TCPWM_PWM_PDL` дозволяє редагувати параметри конфігурації для екземпляру компоненти.

Основна вкладка (Basic).

Ця вкладка містить параметри компоненти, які використовуються в загальних налаштуваннях ініціалізації периферійних пристроїв.

- `PwmMode` – вибір режиму роботи ШІМ.
- `ClockPrescaler` – ділення вхідних тактових імпульсів.
- `RunMode` – якщо вибрано режим неперервного підрахунку, лічильник працює неперервно. Якщо ж вибрано режим однократного виконання, лічильник працює протягом одного періоду і зупиняється.
- `PwmAlignment` – вибір напрямку, в якому виконується підрахунок PWM (`LeftUp`, `Right = Down`, `Center = Up/Down1`, `Asymmetric = Up/Down2`).
- `DeadClocks` – кількість періодів тактової частоти "мертвого" часу між виходами. Діапазон 0-255.
- `Period0` – встановлює період лічильника. Діапазон 0-65535 - для 16-ти бітної роздільної здатності або 0-4294967295 - для 32-х бітної роздільної здатності.
- `EnablePeriodSwap` – якщо встановлений цей прапорець, то періоди будуть змінюватися між нульовим періодом та першим періодом при наступному `ov/un`, коли подія обміну була зареєстрована. В центральному та

асиметричному режимах періоди змінюються місцями тільки при події недостатнього заповнення.

- Period1 – встановлює період лічильника після першого обміну. Діапазон 0-65535 - для 16-ти бітної роздільної здатності або 0-4294967295 - для 32-х бітної роздільної здатності.

- Compare0 – встановлює значення порівняння. Коли значення лічильника рівне значенню порівняння, то вихідні імпульси порівняння мають високий рівень. Діапазон 0-65535 – для 16-ти бітної роздільної здатності або 0-4294967295 - для 32-х бітної роздільної здатності.

- EnableCompareSwar – при виборі регістр порівняння змінюється між порівнянням 0 та порівнянням 1 на наступному ov/up після реєстрації обміну.

- Compare1 – встановлює значення порівняння. Якщо значення лічильника рівне величині порівняння, вихідні імпульси порівняння мають високий рівень.

- InterruptSource – вибирає, які події можуть викликати порівняння.

Вкладка Advanced.

- ReloadInput – визначає чи потрібний вхід перезавантаження і як реєструється вхідний сигнал перезавантаження.

- SwarInput – цей вхід управляє, коли відбувається порівняння і заміна періодів, а також спосіб реєстрації цього вхідного сигналу.

- KillInput – визначає, як поводить себе вхідний сигнал видалення і як цей вхід реєструється.

- StartInput – визначає, чи потрібний вхід запуску і як цей вхід реєструється.

- CountInput – визначає, чи потрібний лічильний вхід і як цей вхід реєструється.

- KillMode – визначає, що сигнал видалення робить з PWM.

- InvertPwm – якщо вибраний цей параметр, то головний вихід PWM інвертується.

InvertPwm_n - якщо вибраний цей параметр, то головний вихід PWM (PWM_n) інвертується.

Основні характеристики компоненти TCPWM_Counter_PDL:

- 16-ти або 32-х розрядні таймер/лічильник (Timer/Counter).
- Програмований регістр періоду.
- Програмований регістр порівняння. Під час події порівняння величину порівняння можна поміняти місцями з буферизованим значенням порівняння.
- Захоплення вхідних даних із буферним регістром.
- Режим сумуючого підрахунку, віднімаючого підрахунку або сумуючого і віднімаючого підрахунку (реверсивний підрахунок).
- Режими неперервного або однократного виконання.
- Переривання та вихід по переповненню, переходу через нуль, захопленню або порівнянню.

Методичні вказівки з курсу “Вбудовані системи опрацювання даних та управління на основі нейронних мереж” 2025 р. Рабик В.

- Запуск, перезавантаження, зупинка, захоплення та підрахунок вхідних даних.

Загальний опис.

Компонента TCPWM_Counter_PDL є графічним об'єктом, що конфігурується, створеним як верхній рівень драйвера cy_tcpwm, доступного в PDL. Вона дозволяє виконувати з'єднання на основі схеми і апаратну конфігурацію, яка визначена в діалоговому вікні "Component Configure".

Компонента TCPWM_Counter_PDL дозволяє швидко конфігурувати апаратне забезпечення TCPWM для функції таймера/лічильника. Ця компонента дозволяє вимірювати часові інтервали або підраховувати зовнішні події.

Компоненту TCPWM_Counter_PDL потрібно використовувати завжди, коли необхідно реалізувати певний часовий інтервал або виконати вимірювання часового інтервалу. Наприклад:

- Реалізація періодичного переривання для виконання інших системних задач.
- Вимірювання частоти вхідного сигналу.
- Вимірювання тривалості імпульсу вхідного сигналу.
- Вимірювання часу між двома зовнішніми подіями.
- Підрахунок подій.
- Запуск інших системних ресурсів після X подій.
- Запис міток часу при виникненні подій.

Швидкий старт роботи з компонентою TCPWM_Counter_PDL.

1. Перетягнути компоненту TCPWM_Counter_PDL з папки Cypress/Digital/Function/ в каталозі компонент на схему проекту (екземпляр цієї компоненти приймає ім'я Counter_1).

2. Двічі клацнути на екземплярі компоненти, щоб відкрити діалогове вікно налаштування (Configure) і налаштувати параметри компоненти згідно з вимогами.

3. В цій компоненті немає внутрішніх джерел тактових імпульсів. Тому потрібно подати на вхід TCPWM_Counter_PDL тактові імпульси з зовнішнього джерела. В компоненті TCPWM_Counter_PDL доступна функція попереднього масштабування тактової частоти для подальшого ділення тактової частоти.

4. Побудувати проект. Для того, щоб перевірити правильність проекту, потрібно додати необхідні модулі PDL в Workspace Explorer та згенерувати задану конфігурацію для екземпляра Counter_1.

5. В файлі main.c, ініціалізувати периферійні пристрої та запустити додаток за допомогою API драйверів та макросів препроцесора компонентів.

Якщо вхідний вивід (наприклад, start) не використовується, для запуску підрахунку потрібно викликати програмну подію Cy_TCPWM_TriggerStart або Cy_TCPWM_TriggerReloadOrIndex. Наприклад:

```
(void)Cy_TCPWM_Counter_Init(Counter_1_HW, Counter_1_CNT_NUM, &Counter_1_config);  
    Cy_TCPWM_Enable_Multiple(Counter_1_HW, Counter_1_CNT_MASK);  
    Cy_TCPWM_TriggerStart(Counter_1_HW, Counter_1_CNT_MASK);
```

Зовнішній вигляд символу компоненти Counter приведено на рис. 3.6.

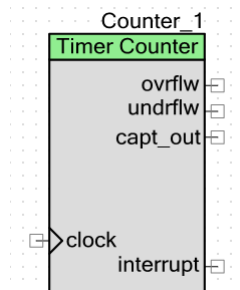


Рис. 3.6. Зовнішній вигляд символу компоненти Counter

Для кожного режиму драйвер TCPWM має структуру конфігурації, функцію ініціалізації, функцію вклучення.

Потрібно вказати параметри конфігурації у структурі даних PWM, структурі даних Counter або структурі даних QuadDec.

Потім потрібно викликати відповідну функцію Init:

```
Cy_TCPWM_PWM_Init,  
Cy_TCPWM_Counter_Init або  
Cy_TCPWM_QuadDec_Init.
```

В цих функціях потрібно вказати адресу використаної структури як параметр. Для дозволу (включення) PWM, Counter або QuadDec потрібно викликати відповідну функцію

```
Cy_TCPWM_PWM_Enable,  
Cy_TCPWM_Counter_Enable або  
Cy_TCPWM_QuadDec_Enable.
```

Функція `Cy_TCPWM_PWM_Init()`.

```
cy_en_tcpwm_status_t Cy_TCPWM_PWM_Init(TCPWM_Type * base,  
                                         uint32_t cntNum,  
                                         cy_stc_tcpwm_pwm_config_t const * config  
                                         )
```

Ця функція ініціалізує лічильник в блоці TCPWM для операції PWM.

Параметри:

- base - вказівник на екземпляр TCPWM.
- cntNum - номер екземпляра лічильника в вибраному TCPWM.
- config - вказівник на структуру конфігурації.

Функція повертає код помилки / статусу.

Функція `Cy_TCPWM_PWM_DeInit()`.

```
cy_en_tcpwm_status_t Cy_TCPWM_PWM_DeInit(TCPWM_Type * base,  
                                           uint32_t cntNum,  
                                           cy_stc_tcpwm_pwm_config_t const * config
```

)
Функція деініціалізує лічильник в блоці TCPWM для операції PWM, повертає значення регістрів за замовчуванням.

Функція `Cy_TCPWM_PWM_Enable`.
`__STATIC_INLINE void Cy_TCPWM_PWM_Enable(TCPWM_Type * base,`
`uint32_t cntNum`
`)`

Ця функція дозволяє роботу (включає) лічильник в блоці TCPWM для операції PWM.

Параметри:

- base - вказівник на екземпляр TCPWM.
- cntNum - номер екземпляра лічильника в вибраному TCPWM.

Функція `Cy_TCPWM_PWM_Disable`.
`__STATIC_INLINE void Cy_TCPWM_PWM_Disable(TCPWM_Type * base,`
`uint32_t cntNum`
`)`

Ця функція відключає лічильник в блоці TCPWM. Параметри цієї функції аналогічні параметрам попередньої функції.

Функція `Cy_TCPWM_PWM_SetCompare0`.
`__STATIC_INLINE void Cy_TCPWM_PWM_SetCompare0(TCPWM_Type * base,`
`uint32_t cntNum,`
`uint32_t compare0`
`)`

Функція встановлює значення порівняння для `Compare0`, якщо дозволений режим порівняння.

Параметри:

- base - вказівник на екземпляр TCPWM.
- cntNum - номер екземпляра лічильника в вибраному TCPWM.
- compare0 - значення `compare0`.

Функція `Cy_TCPWM_PWM_GetCompare0`.
`__STATIC_INLINE uint32_t Cy_TCPWM_PWM_GetCompare0(TCPWM_Type * base,`
`uint32_t cntNum`
`)`

Ця функція повертає значення `compare0`.

Функція `Cy_TCPWM_PWM_SetCompare1`.

Ця функція аналогічна функції `Cy_TCPWM_PWM_SetCompare0`.

Функція `Cy_TCPWM_PWM_GetCompare1`.

Ця функція аналогічна функції `Cy_TCPWM_PWM_GetCompare0`.

Функція `Cy_TCPWM_PWM_SetCounter()` .
`__STATIC_INLINE void Cy_TCPWM_PWM_SetCounter(TCPWM_Type * base,
uint32_t cntNum,
uint32_t count
)`

Ця функція дозволяє встановити значення лічильника.

Функція `Cy_TCPWM_PWM_GetCounter()` .
`__STATIC_INLINE uint32_t Cy_TCPWM_PWM_GetCounter(TCPWM_Type * base,
uint32_t cntNum
)`

Ця функція повертає значення лічильника.

Функція `Cy_TCPWM_PWM_SetPeriod0()` .
`__STATIC_INLINE void Cy_TCPWM_PWM_SetPeriod0(TCPWM_Type * base,
uint32_t cntNum,
uint32_t period0
)`

Ця функція дозволяє встановити величину регістру періоду.

Функція `Cy_TCPWM_PWM_GetPeriod0()` .
`__STATIC_INLINE uint32_t Cy_TCPWM_PWM_GetPeriod0(TCPWM_Type * base,
uint32_t cntNum
)`

Ця функція зчитує величину регістру періоду.

Функція `Cy_TCPWM_PWM_SetPeriod1()` аналогічна функції `Cy_TCPWM_PWM_SetPeriod0()`.

Функція `Cy_TCPWM_PWM_GetPeriod1()` аналогічна функції `Cy_TCPWM_PWM_GetPeriod0()`.

Функція `Cy_TCPWM_PWM_EnablePeriodSwap()` .
`__STATIC_INLINE void Cy_TCPWM_PWM_EnablePeriodSwap(TCPWM_Type * base,
uint32_t cntNum,
bool enable
)`

Ця функція дозволяє заміну періоду через OV і/або UN в залежності від вирівнювання PWM.

Параметри:

- base - вказівник на екземпляр TCPWM.
- cntNum - номер екземпляра лічильника в вибраному TCPWM.
- enable - true – обмін дозволений, false – обмін заборонений.

Функція `Cy_TCPWM_TriggerStart()` .
`__STATIC_INLINE void Cy_TCPWM_TriggerStart(TCPWM_Type * base,
uint32_t counters
)`

Функція ініціює запуск програмного забезпечення для вибраних TCPWMs.
Параметри:

- base - вказівник на екземпляр TCPWM.
- counter - бітове поле, що представляє кожний лічильник в блоці TCPWM.

Функція `Cy_TCPWM_TriggerReloadOrIndex()`.
`__STATIC_INLINE void Cy_TCPWM_TriggerReloadOrIndex(TCPWM_Type * base, uint32_t counters)`

Функція запускає перезавантаження програмного забезпечення або індекс в режимі QuadDec.

Функція `Cy_TCPWM_TriggerStopOrKill()`.
`__STATIC_INLINE void Cy_TCPWM_TriggerStopOrKill(TCPWM_Type * base, uint32_t counters)`

Функція запускає зупинку в режимі Timer Counter або видалення в режимі PWM.

Приклад реалізації проекту з використанням компоненти PWM.

Реалізуємо проект засвічування червоного, синього та зеленого світлодіодів RGB LED стенду PSoC 6 BLE PIONER KIT з використанням компоненти PWM. Алгоритм засвічування світлодіодів зображений на рис. 3.7. $T_{PWM}=1.2$ сек, Compare=0.6 сек.

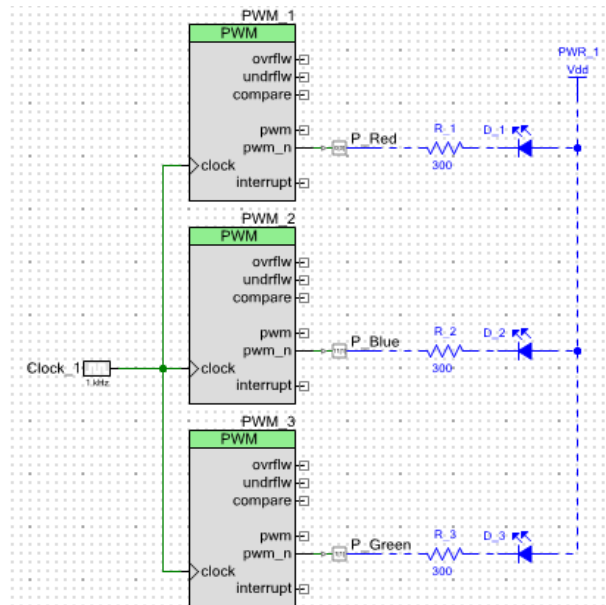


Рис. 3.8. Схема проекту засвічування світлодіодів RGB LED стенду з використанням компонент PWM

Налаштування деяких компонент проекту (рис. 3.9):

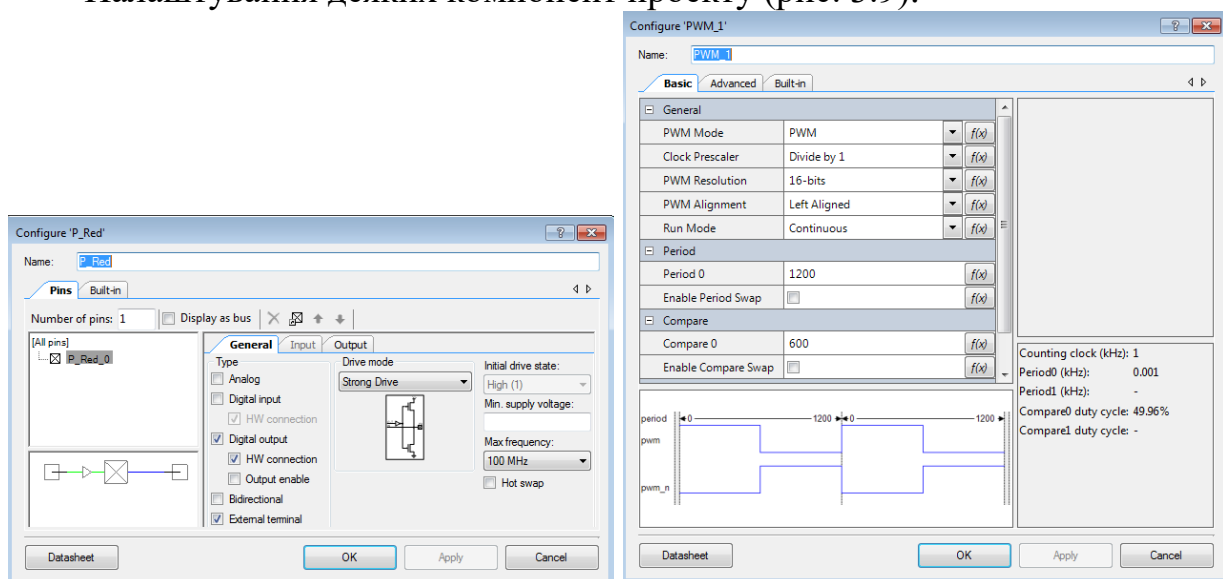


Рис. 3.9. Вигляд вікон налаштування GPIO (P_Red) та PWM (PWM_1)

Вигляд карти виводів мікроконтролера PSoC 6 з назначеними виводами зображено на рис. 3.10.

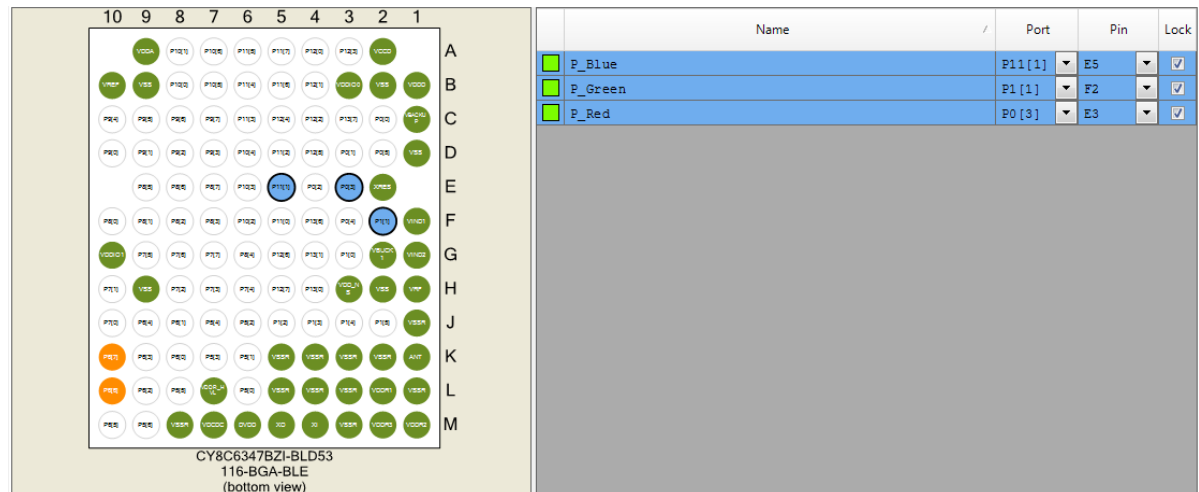


Рис. 3.10. Вигляд карти виводів мікроконтролера PSoC 6 з назначеними виводами

Програмна частина проекту з використанням функцій бібліотеки PDL:

```
// Code LR_3_1
#include "project.h"

int main(void)
{
    for (;;)
    {
        Cy_TCPWM_PWM_Init(PWM_1_HW, PWM_1_CNT_NUM, &PWM_1_config);
        Cy_TCPWM_PWM_Enable(PWM_1_HW, PWM_1_CNT_NUM);
        Cy_TCPWM_TriggerStart(PWM_1_HW, PWM_1_CNT_MASK);
        Cy_TCPWM_Counter_SetCounter(PWM_1_HW, PWM_1_CNT_NUM, 1199);
        CyDelay(1200);

        Cy_TCPWM_PWM_Disable(PWM_1_HW, PWM_1_CNT_NUM);
        Cy_TCPWM_TriggerStopOrKill(PWM_1_HW, PWM_1_CNT_MASK);
        Cy_TCPWM_PWM_DeInit(PWM_1_HW, PWM_1_CNT_NUM, &PWM_1_config);

        Cy_TCPWM_PWM_Init(PWM_2_HW, PWM_2_CNT_NUM, &PWM_2_config);
        Cy_TCPWM_PWM_Enable(PWM_2_HW, PWM_2_CNT_NUM);
        Cy_TCPWM_TriggerStart(PWM_2_HW, PWM_2_CNT_MASK);
        Cy_TCPWM_Counter_SetCounter(PWM_2_HW, PWM_2_CNT_NUM, 1199);
        CyDelay(1200);

        Cy_TCPWM_PWM_Disable(PWM_2_HW, PWM_2_CNT_NUM);
        Cy_TCPWM_TriggerStopOrKill(PWM_2_HW, PWM_2_CNT_MASK);
        Cy_TCPWM_PWM_DeInit(PWM_2_HW, PWM_2_CNT_NUM, &PWM_2_config);

        Cy_TCPWM_PWM_Init(PWM_3_HW, PWM_3_CNT_NUM, &PWM_3_config);
        Cy_TCPWM_PWM_Enable(PWM_3_HW, PWM_3_CNT_NUM);
        Cy_TCPWM_TriggerStart(PWM_3_HW, PWM_3_CNT_MASK);
        Cy_TCPWM_Counter_SetCounter(PWM_3_HW, PWM_3_CNT_NUM, 1199);
        CyDelay(1200);

        Cy_TCPWM_PWM_Disable(PWM_3_HW, PWM_3_CNT_NUM);
        Cy_TCPWM_TriggerStopOrKill(PWM_3_HW, PWM_3_CNT_MASK);
        Cy_TCPWM_PWM_DeInit(PWM_3_HW, PWM_3_CNT_NUM, &PWM_3_config);
    }
}
```

Завдання.

1. Реалізувати проект засвічування червоного, синього та зеленого світлодіодів RGB LED стану з використанням компоненти PWM. В схемі використати кнопку SW_2 та вивід GPIO, до якого вона підключена. Алгоритм засвічування світлодіодів, якщо кнопка SW_2 не натиснута, зображений на рис. 3.11. При натисканні кнопки SW_2 та на протязі її утримання алгоритм засвічування зображений на рис. 3.12. Величина періоду PWM (T_{PWM}), коли кнопка SW_2 не натиснута, складає 1500 мсек. Якщо ж кнопка натиснута, то величина періоду складає 1000 мсек.

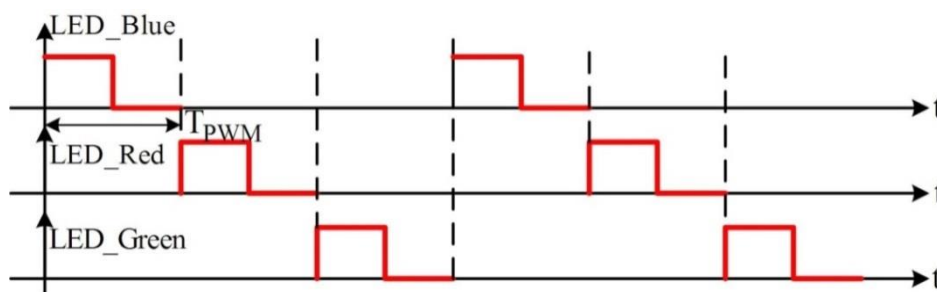


Рис. 3.11. Часові діаграми засвічування світлодіодів RGB LED стану при ненатиснутій кнопці SW_2

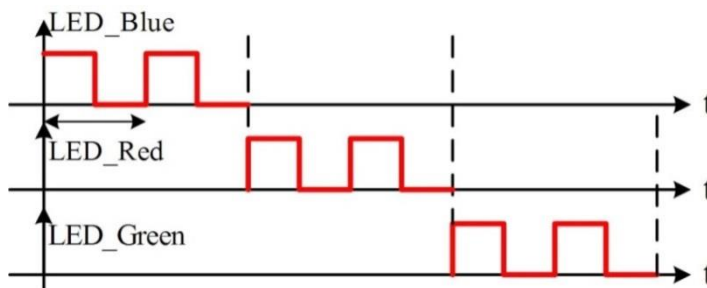


Рис. 3.12. Часові діаграми засвічування світлодіодів RGB LED стану при натиснутій кнопці SW_2

2. Реалізувати проект плавного засвічування/ гасіння світлодіоду LED8 стану PSoC 6 BLE PIONER KIT з використанням схеми, зображеної на рис. 3.13. Алгоритм засвічування світлодіодів зображений на рис. 3.14. Параметри PWM – $T_{PWM} = 1$ мсек, $T = 20$ мсек. Час плавного засвічування LED8 складає 2 сек.

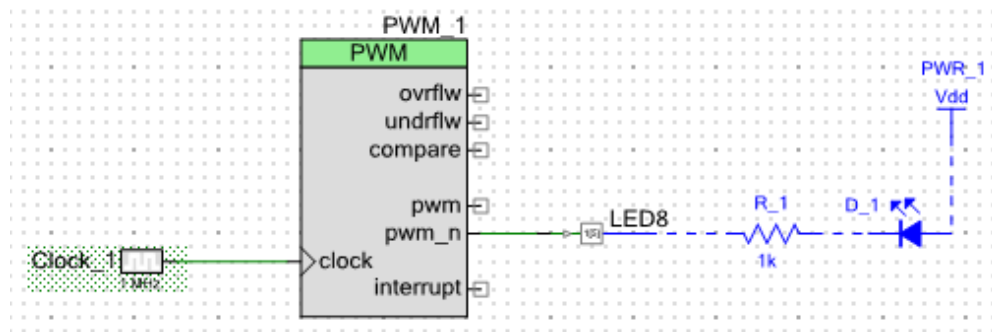


Рис. 3.13. Апаратна частина проекту до завдання 2.

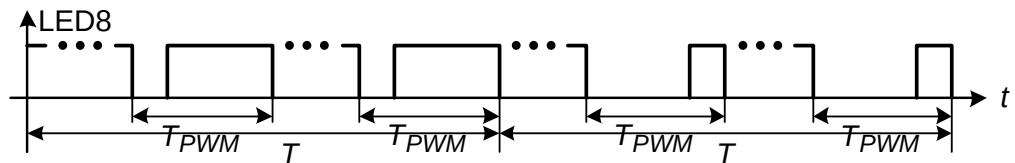


Рис. 3.14. Часові діаграми засвічування світлодіоду LED8 стенду

3. Реалізувати проект засвічування червоного, синього та зеленого світлодіодів RGB LED стенду PSoC 6 BLE PIONER KIT з використанням компонент PWM. Алгоритм засвічування світлодіодів зображений на рис. 3.15. Період PWM (T_{PWM}) складає 2 сек, а $t_{LED_RGB}=12$ сек.

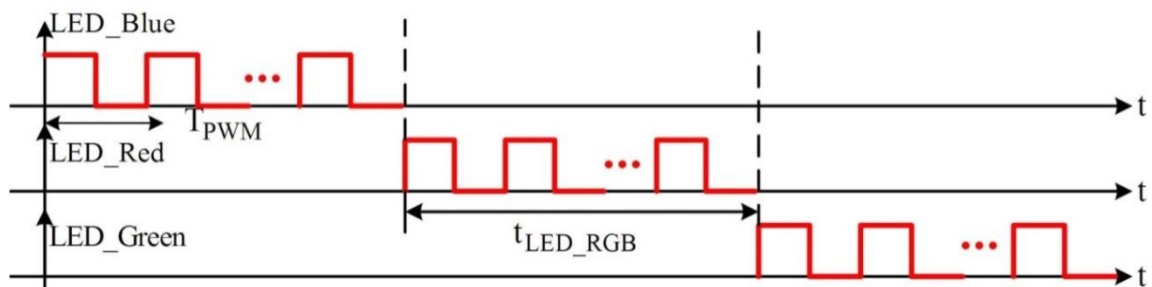


Рис. 3.15. Часові діаграми засвічування світлодіодів RGB LED стенду